FinanSync

Documentación del proyecto en la nube — Google Cloud Platform

Curso	Cloud Computing — Course Project (entrega final)
Universidad	Universidad Galileo — FISICC
Integrantes	Roberto Molina · Luis Cordova
Aplicación	https://finansync.me
Repositorio	github.com/RobertoJavier1/FinanSync-v2
Proyecto GCP	`finansync-74cd1` · región `us-central1`

Noviembre 2026

Contenido

1. Descripción del proyecto
2. Arquitectura en la nube
3. Servicios utilizados y su integración
4. Flujos principales
5. Pasos de despliegue
6. Consideraciones de seguridad
7. Estimación de costos
8. Escalabilidad y uso de recursos

1. Descripción del proyecto

FinanSync es una aplicación web de finanzas personales. Cada usuario registra sus ingresos y gastos, define presupuestos por categoría y metas de ahorro, y ve gráficas y reportes de su actividad. Sobre esa base se agregan funciones que usan servicios de Google Cloud:

- **Lectura de facturas:** el usuario sube o fotografía una factura y la app llena la transacción sola. Cloud Vision extrae el texto, Gemini (en Vertex AI) lo interpreta y sugiere la categoría, y la imagen queda guardada en un bucket privado de Cloud Storage.
- **Asistente financiero:** un chat y una sección de perspectivas, ambos con Gemini en Vertex AI, que analizan los movimientos del usuario.
- **Estado de cuenta mensual:** el día 1 de cada mes, Cloud Scheduler dispara un proceso que arma el PDF del mes de cada usuario y se lo envía por correo.

La aplicación está hecha con **Next.js 15** (React y TypeScript) y corre como un contenedor en **Cloud Run**, con el dominio propio `finansync.me`. Cada push a la rama `main` la vuelve a desplegar sola mediante **Cloud Build**.

Servicios externos (no cuentan dentro de los 8)

Servicio	Uso
Supabase	Autenticación (correo y Google) y base de datos Postgres con Row Level Security. El profesor autorizó mantener la base de datos en Supabase, con la condición de que no cuente como uno de los servicios de nube.
Resend	Envío del correo con el estado de cuenta mensual.
Namecheap	Registro del dominio `finansync.me`, que apunta a Cloud Run.

2. Arquitectura en la nube

Todos los servicios de Google Cloud están en el proyecto `finansync-74cd1` y en la región `us-central1`. Cloud Run es el centro de la arquitectura: recibe el tráfico de los usuarios y la llamada mensual de Cloud Scheduler, y desde ahí se consumen Storage, Vision y Vertex AI. Cloud Build arma y publica cada versión nueva, y Cloud IAM define con qué identidad corre cada pieza.

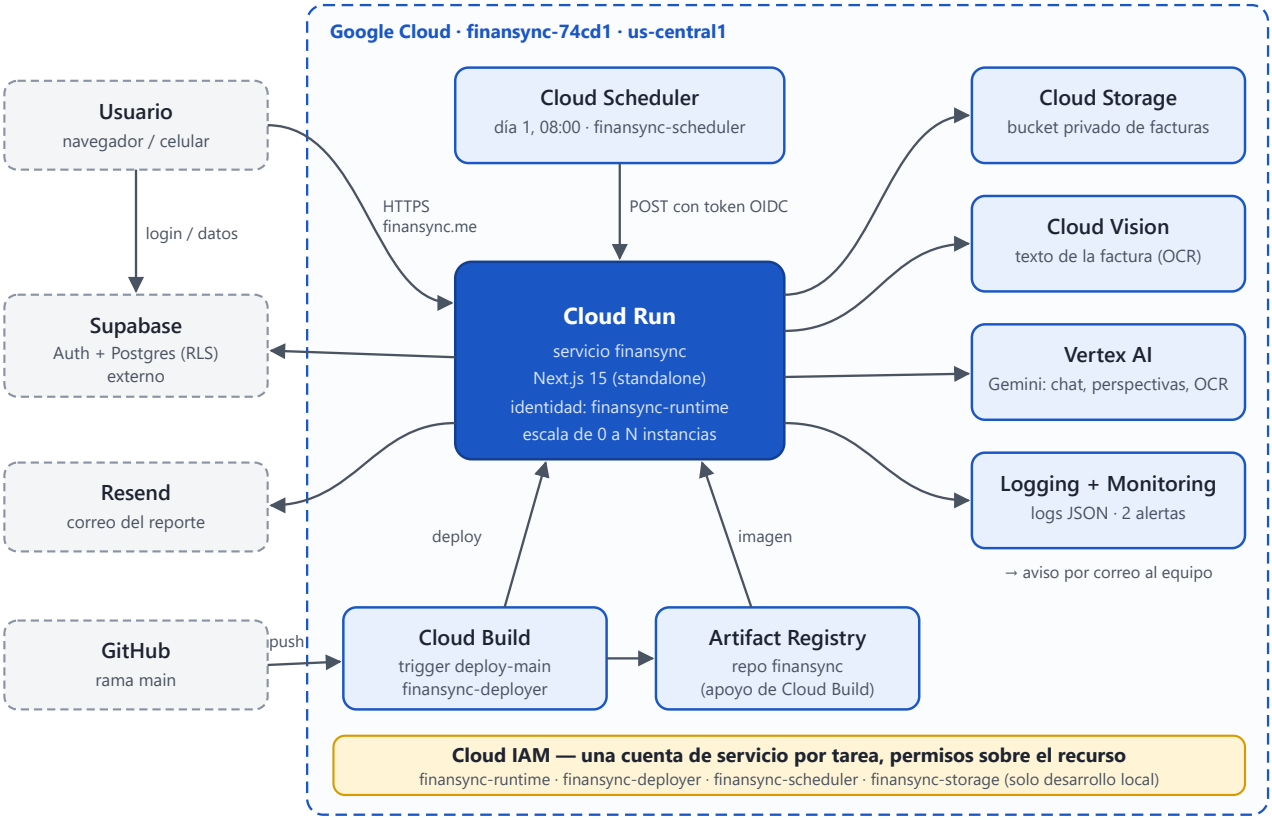


Figura 1. Arquitectura de FinanSync en Google Cloud. En azul, los servicios de GCP; con borde punteado, los servicios externos.

3. Servicios utilizados y su integración

El proyecto usa ocho servicios de Google Cloud. Ninguno es de Firebase: todos se configuran desde la consola o con `gcloud`, y la app los consume con las librerías oficiales de Google Cloud.

#	Servicio	Categoría	Rol en el proyecto y cómo se integra	Dónde se ve
1	**Cloud Run**	Cómputo	Ejecuta la app como un contenedor (imagen multi-etapa `node:20-alpine` con la salida `standalone` de Next.js). Atiende tanto las páginas como las rutas `/api`. Escala a cero cuando no hay tráfico y corre con la identidad `finansync-runtime`. Tiene mapeado el dominio `finansync.me` con certificado HTTPS administrado por Google.	`Dockerfile`, `next.config.ts`
2	**Cloud Build**	CI/CD	El trigger `deploy-main`, conectado al repositorio de GitHub, se dispara con cada push a `main`. Construye la imagen etiquetada con el hash corto del commit, la sube a Artifact Registry y despliega una revisión nueva en Cloud Run. Corre con la cuenta `finansync-deployer`.	`cloudbuild.yaml`
3	**Cloud Storage**	Almacenamiento	El bucket `finansync-facturas` guarda las imágenes de las facturas con la ruta `<usuario>/<transacción>.<ext>`. Es privado (acceso uniforme y prevención de acceso público). Para mostrar una factura, la app firma una URL que vence en 15 minutos, y solo después de verificar que la transacción es del usuario.	`lib/gcs.ts`, `app/api/facturas/`

4	**Cloud Vision API**	IA	Primer paso de la lectura de facturas: extrae el texto de la imagen con `documentTextDetection`. Así a Gemini solo se le manda texto, lo que reduce el consumo de tokens.	`lib/vision.ts`
5	**Vertex AI**	IA	Da acceso a Gemini para interpretar el texto de la factura (monto, fecha, comercio y categoría sugerida), para el chat financiero y para las perspectivas. Usa `gemini-3.8-flash` y, si ese modelo responde 429, pasa a `gemini-3.5-flash`.	`lib/vertex.ts`, `app/api/chat`, `app/api/perspectivas`
6	**Cloud Scheduler**	Automatización	El job `estado-cuenta-mensual` (`0 8 1 * *`, zona America/Guatemala) hace un POST a `/api/reportes/mensual` con un token OIDC firmado por `finansync-scheduler`. El endpoint verifica la firma y que el token sea de esa cuenta, y luego arma y envía el PDF de cada usuario.	`app/api/reportes/mensual/route.ts`
7	**Cloud IAM**	Seguridad	Cuatro cuentas de servicio propias, cada una con roles no predeterminados y acotados al recurso que necesita (detalle en la sección 6). En producción no hay llaves: Cloud Run obtiene las credenciales de su identidad (Application Default Credentials).	`lib/google-auth.ts`
8	**Cloud Logging y Monitoring**	Observabilidad	La app escribe logs en JSON de una línea con los campos `evento` y `severity`, que Cloud Logging indexa. Hay dos políticas de alerta con aviso por correo: fallos o rechazos del reporte mensual, y más de cinco respuestas 5xx en cinco minutos.	`monitoring/`

> Además se usa **Artifact Registry** (repositorio Docker `finansync`) para guardar las imágenes que produce Cloud Build. Es un servicio de apoyo del pipeline y no lo contamos dentro de los ocho.

4. Flujos principales

4.1 Lectura de una factura

1. El usuario sube o fotografía la factura en la pantalla de transacciones.
2. `POST /api/facturas` valida la sesión en el servidor y manda la imagen a **Cloud Vision**, que devuelve el texto.
3. El texto se envía a **Gemini en Vertex AI**, que devuelve en JSON el monto, la fecha, el comercio y la categoría sugerida.
4. El usuario revisa el formulario ya lleno y guarda la transacción en Supabase.
5. `POST /api/facturas/subir` comprueba que la transacción sea del usuario y sube la imagen a **Cloud Storage**.
6. Para verla después, `/api/facturas/ver` devuelve una URL firmada de 15 minutos.

4.2 Despliegue continuo

1. Un integrante hace push a `main` en GitHub.
2. El trigger `deploy-main` de **Cloud Build** ejecuta `cloudbuild.yaml`: *docker build* (con las variables públicas de Supabase como `--build-arg`), *docker push* a Artifact Registry y *gcloud run deploy*.
3. **Cloud Run** crea una revisión nueva y le pasa el tráfico. Si la revisión no arranca, el tráfico se queda en la anterior.

4.3 Estado de cuenta mensual

1. El día 1 a las 08:00, **Cloud Scheduler** llama a `/api/reportes/mensual` con un token OIDC de `finansync-scheduler`.
2. El endpoint verifica el token. Si lo rechaza, escribe `reporte_mensual_rechazado` con severidad ERROR.
3. Para cada usuario que tiene el reporte activado, arma el PDF del mes y lo envía con Resend. Si falla un usuario, los demás siguen.
4. Al terminar escribe `reporte_mensual_fin` con los conteos. Si hubo errores, **Cloud Monitoring** manda un correo al equipo.

5. Pasos de despliegue

Los comandos se ejecutan en Cloud Shell. Se asume que existe el proyecto `finansync-74cd1` con facturación activa. Para abreviar:

```
PROJECT=finansync-74cd1
REGION=us-central1
SA=iam.gserviceaccount.com
gcloud config set project $PROJECT
```

Paso 1 — Habilitar las APIs

```
gcloud services enable run.googleapis.com cloudbuild.googleapis.com \
  artifactregistry.googleapis.com storage.googleapis.com vision.googleapis.com \
  aiplatform.googleapis.com cloudscheduler.googleapis.com \
  logging.googleapis.com monitoring.googleapis.com
```

Paso 2 — Crear las cuentas de servicio

```
for c in finansync-runtime finansync-deployer finansync-scheduler finansync-storage; do
  gcloud iam service-accounts create $c
done
```

Paso 3 — Crear el bucket de facturas

```
gcloud storage buckets create gs://finansync-facturas --location=$REGION \
  --uniform-bucket-level-access --public-access-prevention

# acceso solo sobre este bucket, no sobre el proyecto
for c in finansync-runtime finansync-storage; do
  gcloud storage buckets add-iam-policy-binding gs://finansync-facturas \
    --member=serviceAccount:$c@$PROJECT.$SA --role=roles/storage.objectAdmin
done
```

Paso 4 — Permisos de la cuenta de la app (`finansync-runtime`)

```
for r in roles/aiplatform.user roles/logging.logWriter roles/monitoring.metricWriter; do
  gcloud projects add-iam-policy-binding $PROJECT \
    --member=serviceAccount:finansync-runtime@$PROJECT.$SA --role=$r
done

# necesario para firmar las URL de las facturas sin llave privada
gcloud iam service-accounts add-iam-policy-binding finansync-runtime@$PROJECT.$SA \
  --member=serviceAccount:finansync-runtime@$PROJECT.$SA \
  --role=roles/iam.serviceAccountTokenCreator
```

Paso 5 — Artifact Registry y permisos del pipeline

```
gcloud artifacts repositories create finansync --repository-format=docker --location=$REGION

D=serviceAccount:finansync-deployer@$PROJECT.$SA
gcloud projects add-iam-policy-binding $PROJECT --member=$D --role=roles/run.developer
gcloud projects add-iam-policy-binding $PROJECT --member=$D --role=roles/logging.logWriter
gcloud artifacts repositories add-iam-policy-binding finansync --location=$REGION \
  --member=$D --role=roles/artifactregistry.writer
gcloud iam service-accounts add-iam-policy-binding finansync-runtime@$PROJECT.$SA \
  --member=$D --role=roles/iam.serviceAccountUser
```

Paso 6 — Primer despliegue en Cloud Run

El primer despliegue define la identidad del servicio y sus variables de entorno. Los siguientes los hace Cloud Build y conservan esa configuración.

```
gcloud builds submit --config=cloudbuild.yaml \
  --substitutions=SHORT_SHA=inicial,_SUPABASE_URL=...,_SUPABASE_ANON_KEY=...

gcloud run deploy finansync --region=$REGION \
  --image=$REGION-docker.pkg.dev/$PROJECT/finansync/app:inicial \
  --service-account=finansync-runtime@$PROJECT.$SA \
  --allow-unauthenticated \
  --set-env-vars=GCS_BUCKET_NAME=finansync-facturas,REPORTE_SERVICE_ACCOUNT=finansync-
scheduler@$PROJECT.$SA \
  --update-env-
vars=SUPABASE_SERVICE_ROLE_KEY=...,RESEND_API_KEY=...,CORREO_REMITENTE=...,REPORTE_CRON_TOKEN=...
```

> En Cloud Run no se configuran las variables `GCP_RUNTIME_*` ni `GCS_*` con llaves. Sin ellas, las librerías de Google usan la identidad del servicio. Esas variables solo existen en el `.env.local` de desarrollo.

Paso 7 — Trigger de Cloud Build

En la consola: *Cloud Build → Activadores → Crear activador*.

- Evento: push a la rama `^main$` del repositorio `RobertoJavier1/FinanSync-v2` (conexión con la GitHub App).
- Configuración: archivo `cloudbuild.yaml`.
- Sustituciones: `_SUPABASE_URL` y `_SUPABASE_ANON_KEY`.
- Cuenta de servicio: `finansync-deployer`.

Paso 8 — Dominio propio y login con Google

```
gcloud beta run domain-mappings create --service=finansync --domain=finansync.me --region=$REGION
```

- En Namecheap se crean los registros A y AAAA que indica el comando. Google emite el certificado HTTPS solo.
- En el cliente OAuth de Google Cloud se agrega `https://finansync.me` como origen autorizado de JavaScript.

- En Supabase (*Authentication → URL Configuration*) se agregan la Site URL y la URL de redirección `https://finansync.me/auth/callback`.

Paso 9 — Cloud Scheduler

```
gcloud run services add-iam-policy-binding finansync --region=$REGION \
  --member=serviceAccount:finansync-scheduler@$PROJECT.$SA --role=roles/run.invoker

gcloud scheduler jobs create http estado-cuenta-mensual --location=$REGION \
  --schedule="0 8 1 * *" --time-zone="America/Guatemala" \
  --uri="https://finansync.me/api/reportes/mensual" --http-method=POST \
  --oidc-service-account-email=finansync-scheduler@$PROJECT.$SA
```

Para probarlo sin esperar al día 1: `gcloud scheduler jobs run estado-cuenta-mensual --location=$REGION`.

Paso 10 — Alertas de Monitoring

```
gcloud beta monitoring channels create --display-name="Correo del equipo" \
  --type=email --channel-labels=email_address=CORREO --format="value(name)"

# copiar el id del canal en notificationChannels de los dos JSON, y luego:
gcloud alpha monitoring policies create --policy-from-file=monitoring/alerta-reporte-mensual.json
gcloud alpha monitoring policies create --policy-from-file=monitoring/alerta-errores-5xx.json
```

Paso 11 — Verificación

- Abrir `https://finansync.me`, iniciar sesión con Google y registrar una transacción con factura.
- Revisar que la imagen aparezca en el bucket y que se pueda ver desde la app.
- Forzar el job de Scheduler y buscar `jsonPayload.evento="reporte_mensual_fin"` en Cloud Logging.
- Llamar al endpoint con `Authorization: Bearer token-falso`: tiene que devolver 401 y, a los pocos minutos, llegar el correo de la alerta.

6. Consideraciones de seguridad

6.1 Identidades y permisos (Cloud IAM)

Cada pieza usa su propia cuenta de servicio y solo con los permisos que necesita. Cuando el rol se puede dar sobre el recurso (bucket, repositorio, servicio o cuenta) se da ahí y no sobre todo el proyecto.

Cuenta	La usa	Roles
`finansync-runtime`	Cloud Run (la app)	*Proyecto:* Usuario de Vertex AI, Escritor de logs, Escritor de métricas. *Bucket de facturas:* Storage Object Admin. *Sobre sí misma:* Creador de tokens (para firmar URL sin llave).
`finansync-deployer`	Cloud Build	*Proyecto:* Desarrollador de Cloud Run, Escritor de logs. *Repositorio `finansync`:* Escritor de Artifact Registry. *Sobre `finansync-runtime`:* Usuario de cuenta de servicio (para desplegar con esa identidad).
`finansync-scheduler`	Cloud Scheduler	*Servicio `finansync`:* Invocador de Cloud Run. La app no puede firmar tokens a su nombre.
`finansync-storage`	Desarrollo local	*Bucket de facturas:* Storage Object Admin. Es la única cuenta con llave JSON, y esa llave solo está en el `.env.local` de los integrantes.

6.2 Secretos y credenciales

- En producción no hay llaves de cuentas de servicio: Cloud Run obtiene las credenciales de su identidad.
- `.env.local` está en `.gitignore` y nunca se versiona.
- La `service_role` key de Supabase solo existe como variable de entorno del servidor y nunca lleva el prefijo `NEXT_PUBLIC_`. La clave `anon` sí es pública por diseño, porque el acceso a los datos lo limita RLS.

6.3 Datos y aplicación

- **Row Level Security:** las cinco tablas tienen RLS con una política por operación (20 en total), así que cada usuario solo alcanza sus propias filas.
- **Facturas:** el bucket es privado, cada imagen se sirve con una URL firmada de 15 minutos y antes se verifica que la transacción sea del usuario.
- **Rutas de IA:** validan la sesión dentro de la propia ruta con `getUser()` y no dependen solo del middleware.
- **Job mensual:** solo acepta el token OIDC de `finansync-scheduler`. Si la variable `REPORTE_SERVICE_ACCOUNT` falta, rechaza cualquier token, y cada rechazo queda registrado y dispara una alerta.
- **Cabeceras HTTP:** `Strict-Transport-Security`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff` y `Referrer-Policy`.

- **Contenedor:** el proceso corre como el usuario `node`, no como root, y la imagen final solo lleva la salida compilada.
- **Login:** el callback de OAuth solo redirige a rutas internas, lo que evita un open redirect.

6.4 Riesgos conocidos y mejoras futuras

- La cuenta de servicio predeterminada de Compute conserva el rol Editor que GCP le da al crear el proyecto. Ningún servicio de FinanSync la usa; se puede quitar ese rol.
- Los secretos están como variables de entorno de Cloud Run. Pasarlos a Secret Manager agregaría control de acceso y rotación.
- Las rutas de IA no tienen un límite de uso por usuario. Un límite evitaría abusos que eleven el costo de Vertex AI.

7. Estimación de costos

La estimación se hizo con la **Google Cloud Pricing Calculator** (precios en dólares, vigentes al 7 de noviembre de 2026). Se presentan dos escenarios: el uso actual de la app y una proyección a **50,000 usuarios activos al mes**, que es la que sirve para juzgar si la arquitectura es viable al crecer.

7.1 Escenario actual

Hoy la app la usan el equipo y un grupo pequeño de usuarios de prueba. A esa escala, casi todo cabe en las cuotas gratuitas mensuales de GCP: las solicitudes y el tiempo de CPU de Cloud Run, los minutos de Cloud Build, las primeras 1,000 unidades de Cloud Vision, hasta 3 jobs de Cloud Scheduler y los primeros 50 GiB de Cloud Logging. Lo único que se cobra desde el primer uso son los tokens de Vertex AI, unos pocos megabytes en Cloud Storage y Artifact Registry. **El costo actual es menor a US$1 al mes.**

7.2 Proyección a 50,000 usuarios

Supuestos de uso por usuario al mes, de donde salen las cantidades de la calculadora:

Supuesto	Por usuario	Total mensual
Solicitudes a la app (páginas y API)	≈ 300	15.2 millones
Facturas escaneadas	5	250,000 (Vision y Storage)
Tokens de entrada a Gemini (chat, perspectivas, facturas)	≈ 54,000	2,700 millones
Tokens de salida de Gemini	≈ 15,600	780 millones
Imágenes de facturas guardadas (acumulado)	≈ 26 MB	1,300 GB
Tráfico de salida a internet	≈ 12 MB	605 GB
Cómputo en Cloud Run	—	equivalente a 1 vCPU y 0.5 GiB activos todo el mes

Servicio	Concepto	US$/mes
Vertex AI	Gemini 3.8 Flash — 2,700 M tokens de entrada × US$1.50 por millón	4,049.99
	Gemini 3.8 Flash — 780 M tokens de salida × US$7.50 por millón	5,850.04
Cloud Vision	250,000 detecciones de texto (las primeras 1,000 gratis)	373.50
Cloud Run	CPU (facturación por solicitud)	63.07
	Memoria	3.29
	15.2 M solicitudes (los primeros 2 M gratis)	5.28
Red	605 GB de salida a internet (América → América)	67.49

Cloud Storage	1,300 GB Standard en us-central1	25.90
	250,000 operaciones clase A (subidas)	1.23
	250,000 operaciones clase B (lecturas)	0.08
Cloud Logging	20 GiB de logs (dentro de los 50 GiB gratis) más su retención	0.20
Artifact Registry	2 GB de imágenes	0.15
Cloud Build	300 minutos de build (dentro de la cuota gratuita)	0.00
Cloud Scheduler, IAM y Monitoring	1 job (gratis hasta 3), IAM sin costo, métricas de Cloud Run y 2 alertas: costo despreciable. No se incluyeron en la calculadora.	≈ 0.00
Total estimado		**10,440.22**
Costo por usuario al mes		**≈ 0.21**

> **Sobre el modelo de IA.** La calculadora se configuró con Gemini 3.5 Flash (US$1.50 de entrada y US$9.00 de salida por millón de tokens), lo que da un total de **US$11,610.23**. La app usa **Gemini 3.8 Flash** como modelo principal y deja 3.5 Flash solo como respaldo cuando el principal responde 429, así que la tabla recalcula Vertex AI con el precio estándar de 3.8 Flash (US$1.50 / US$7.50). Ese es el precio que rige desde el 1 de enero de 2027. Hasta el 31 de diciembre de 2026, Google lo ofrece a mitad de precio (US$0.75 / US$3.75), con lo que el total del mes sería de unos **US$5,490**.

7.3 Análisis

- **La IA es casi todo el costo.** Vertex AI suma US$9,900 de los US$10,440 (95 %). La infraestructura que corre la app (Cloud Run, Storage, red, logs y pipeline) cuesta unos US$167 al mes, y Cloud Vision otros US$374 para 50,000 usuarios. Es decir, la arquitectura serverless escala de forma barata y el costo depende de cuánto se use el asistente.
- **Viabilidad.** A unos US$0.21 por usuario al mes, la app no se sostiene gratis a esa escala. Es viable con un plan de pago modesto (por ejemplo, funciones de IA ilimitadas solo para usuarios premium) o limitando el uso de IA en el plan gratuito.
- **Formas de bajar el costo:**
  - Un límite mensual de mensajes de chat y escaneos por usuario. Esa misma medida aparece como mejora de seguridad en la sección 6.4.
  - Enviar menos contexto en cada llamada al chat, porque los tokens de entrada son la mitad del gasto en IA.
  - Reutilizar las perspectivas ya generadas mientras no cambien los movimientos del usuario, en vez de pedirlas otra vez.
  - Una regla de ciclo de vida en el bucket que pase las facturas antiguas a Nearline o Coldline, porque el almacenamiento crece cada mes.
- **Control del gasto:** se recomienda una alerta de presupuesto en Cloud Billing para que el equipo reciba un aviso al 50 %, 90 % y 100 % del monto previsto.
- **Fuera de GCP:** la proyección no incluye Supabase, Resend ni la renovación del dominio (unos US$20 al año), que a esta escala tendrían su propio plan de pago.

8. Escalabilidad y uso de recursos

- **Escala automática:** Cloud Run agrega instancias según la cantidad de solicitudes y baja a cero cuando no hay tráfico. La app no guarda estado en la instancia (la sesión está en Supabase y los archivos en Cloud Storage), así que cualquier instancia puede atender cualquier solicitud.
- **Pago por uso:** todos los servicios usados se cobran por consumo. Sin tráfico, el costo de cómputo es prácticamente cero.
- **Menos consumo de IA:** separar la lectura de facturas en dos pasos (Vision extrae el texto y Gemini lo interpreta) reduce los tokens enviados al modelo. Además, si un modelo está saturado, la app cambia a otro.
- **Imágenes livianas:** la imagen del contenedor es multi-etapa y solo lleva la salida `standalone`, lo que acelera el arranque en frío y el despliegue.
- **Despliegues seguros:** cada commit genera una revisión inmutable y etiquetada en Cloud Run, así que volver a una versión anterior es pasarle el tráfico a la revisión previa.
- **Operación:** las alertas de Monitoring avisan por correo de errores 5xx y de fallos del job mensual, sin tener que revisar los logs a mano.